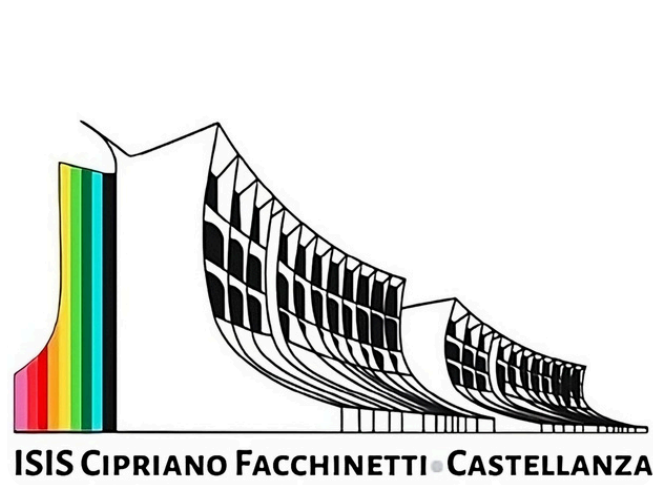
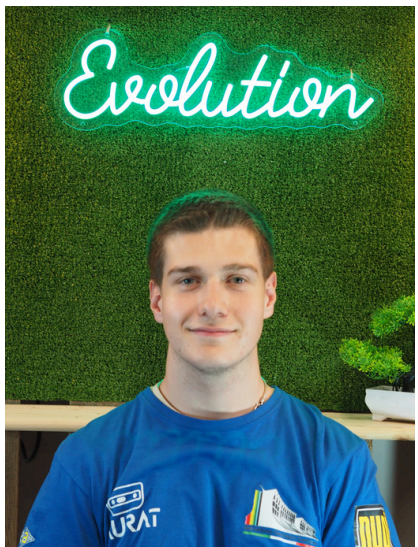




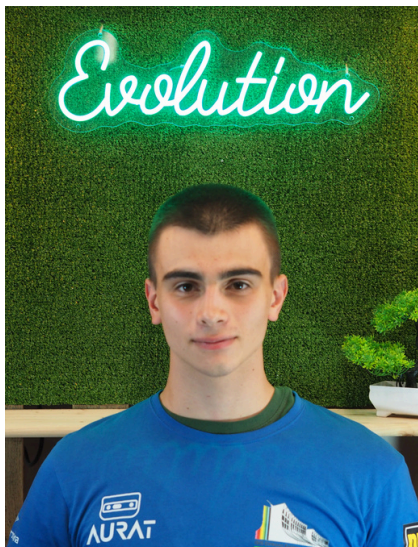
CAP.MARCO MORETTI
SOFTWARE ENGINEER



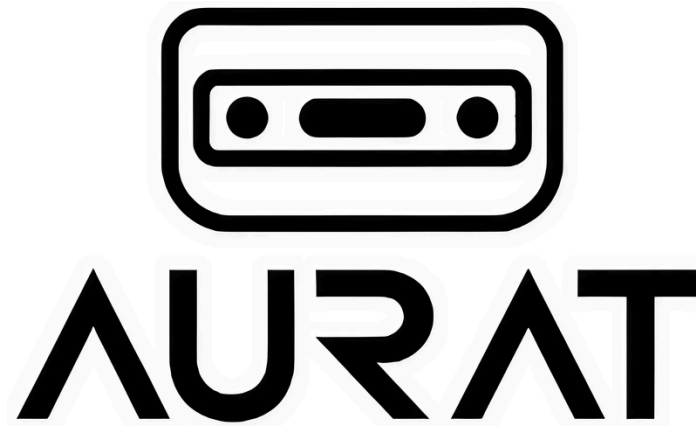
LORIS BERRA
ELETTRONIC



SANDRA D'ARRIGO
ARDUINO ENGINEER

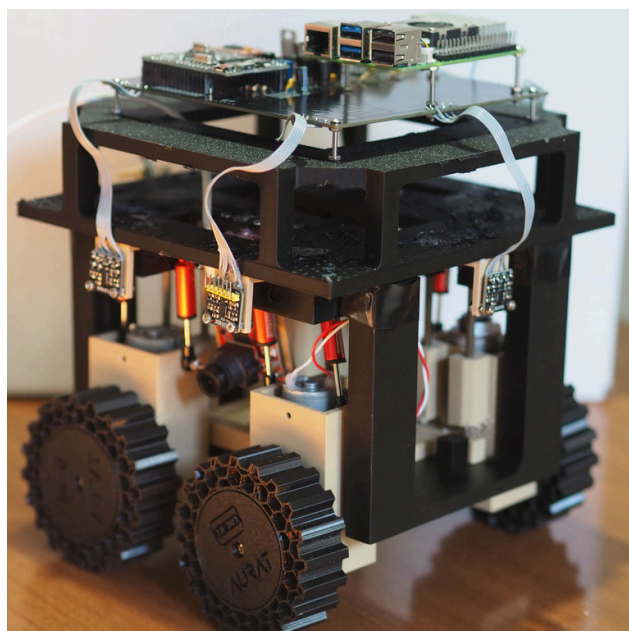


FEDERICO BARBAN
DESIGNER 3D



TEAM

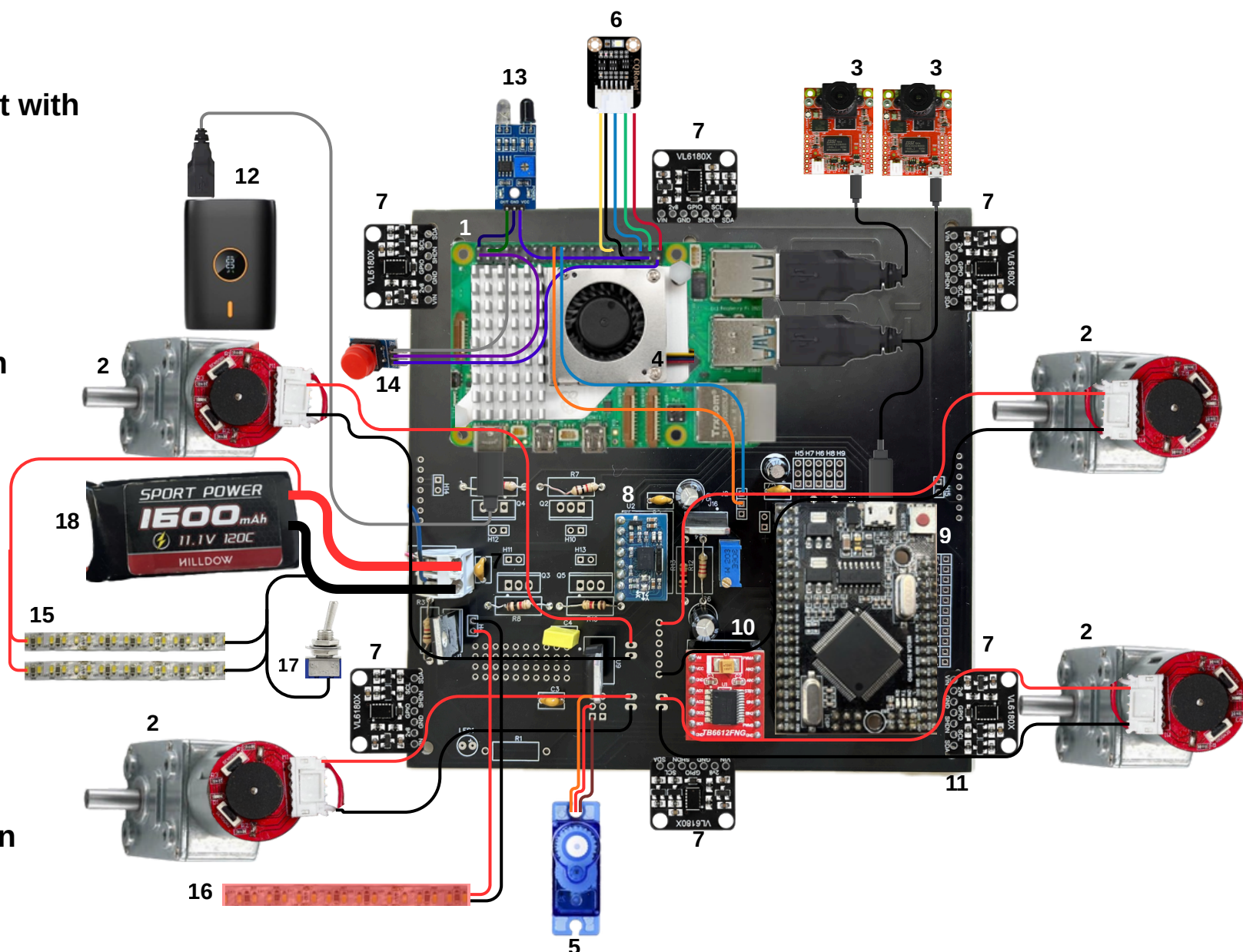
Our team brings together students from **electronics** and **computer science**, each contributing specialized skills that strengthen the project. **Loris Berra**, an electronics student with three years of **RoboCup** experience across multiple categories, has worked on industrial projects at **STMicroelectronics** and **Pomini**, and developed sensor-based safety systems using **ultrasound** and **YOLO-based AI**. He handled the **electronic integration** and **PCB development**. **Marco Moretti**, computer science student and **Lead Software Engineer**, specializes in **Python**, **AI**, **mapping**, **pathfinding**, and **computer vision**. He competed in the **2025 Italian Rescue Maze Championship**, the **2025 OnStage Advanced World Cup** - earning recognition for **best technical documentation** - and the **2026 Italian Rescue Maze Championship**, finishing 5th. **Sandra D'Arrigo**, computer science student and **Arduino specialist**, competed in the **2025** and **2026 Italian Rescue Maze Championships** and contributed to the **visual design** and **presentation materials** for the **2025 World Cup**. **Federico Barban**, computer science student and **mechanical/3D modeling specialist**, has been active in **3D design** since age 12 and created all robot components from scratch. This is his **first robotics competition**. The strength of our team lies in the **diversity of our skills** - electronics, software, mechanics, and design. Each member brings a different perspective, and this variety allows us to **solve problems more effectively**, **learn from one another**, and continuously improve. Collaboration is essential: by combining our individual strengths, we transform **complex challenges** into **achievable goals** and build a more **complete, innovative robot**.



COMPONENTS

- 1.1x Raspberry Pi 5 (8GB): Handles high-level processing
- 2.4x Motors: JGY-370 DC 12V 100 RPM: Precise movement with encoders
- 3.2x OpenMV Cam H7 Plus: For target & possible letter recognition
- 4.1x Raspberry Pi Active Cooler: Cool down Raspy
- 5.1x 9G Micro Servo Motor: Handles kits deployment
- 6.1x TCS34725FN RGB Color Sensor: Tile type recognition
- 7.6x VL6180X ToF Sensors: Measure distance from walls
- 8.1x BNO055 Gyroscope: For precise rotations & straight movements
- 9.1x Arduino Mega Dollatek PRO ATmega USB CH340G: Manages low-level hardware operations
- 10.1x TB6612FNG Motor Driver Board: Handle motors
- 11.1x PCB: Connect components while reducing cables
- 12.1x XHC-N 84 Power Bank: Charge the Raspy
- 13.1x MOD-HW-201 Infrared Sensor: Identify checkpoints
- 14.1x HALJIA Red Push Button: Trigger a LoP
- 15.2x LED Strips: Illuminates the walls for better recognition
- 16.1x LED Strip: Report victims to the referee
- 17.1x Mini Toggle Switch: Cut the power
- 18.1x Sport Power 1600 mAh 11.1 V 120C Battery: Powers the motors & the LED strips

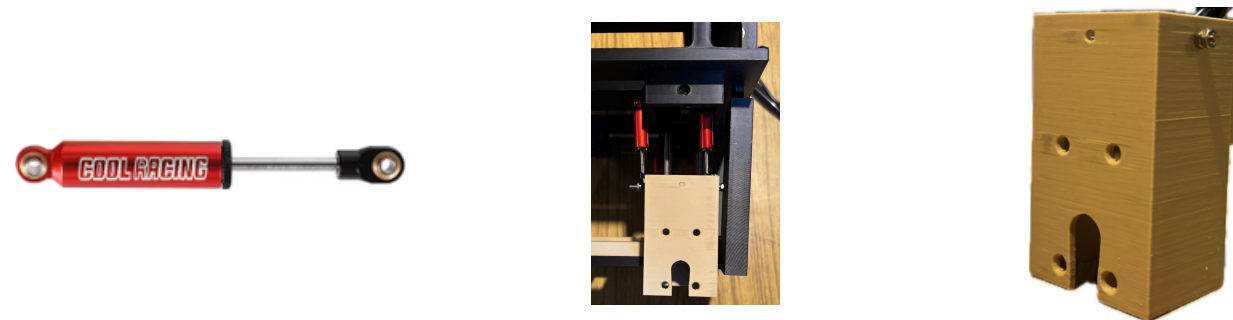
HARDWARE



INNOVATIONS



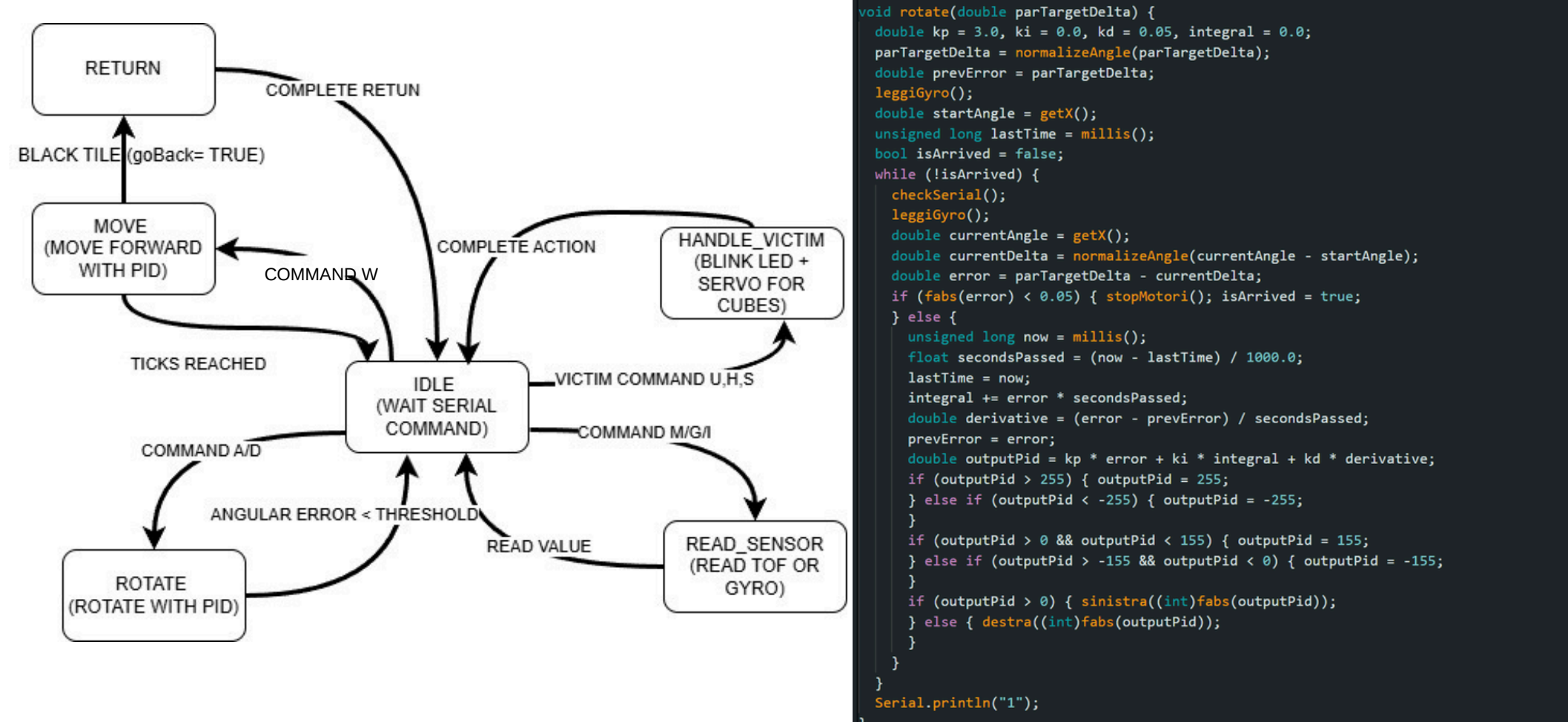
The robot uses **8 cm wheels** equipped with a **toothed TPU tread** that ensures excellent **traction** on ramps, obstacles, and uneven terrain. The tires are made entirely of **TPU**, a flexible and durable material chosen for its superior **grip** and ability to maintain consistent contact **with the ground**. This combination improves stability during movement, enhances **obstacle-handling performance**, and contributes to smoother, more **reliable locomotion** across irregular surfaces.



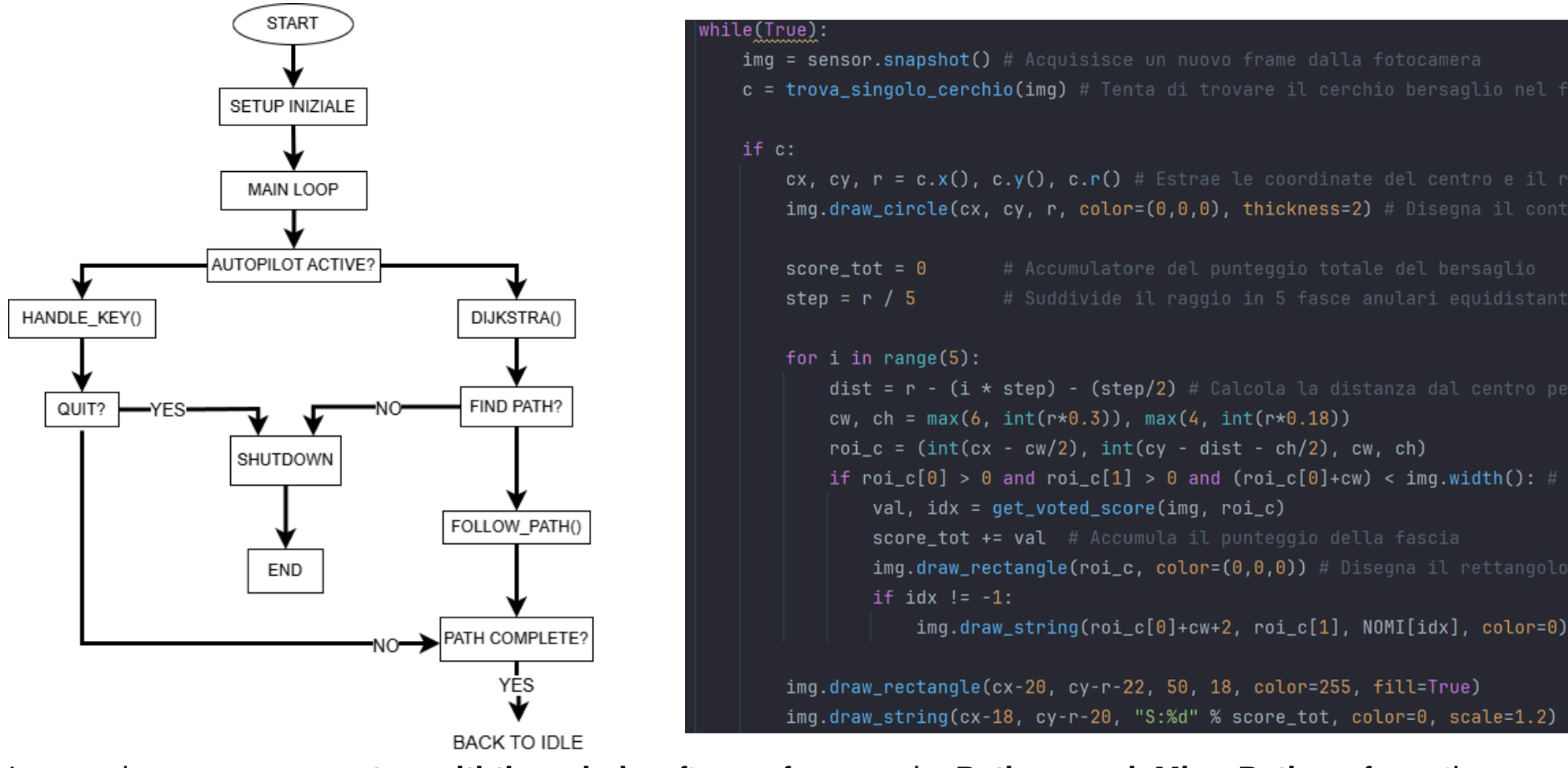
Each motor is mounted on a dedicated **suspension system** that absorbs **terrain irregularities**, reduces **vibrations**, and maintains consistent **wheel-to-ground contact**. The system helps the robot keep traction even when crossing uneven surfaces or small obstacles. The **lubricated rods** further minimize **friction** and mechanical **oscillations**, improving smoothness during acceleration and rotation. This setup increases overall **stability**, protects the mechanical components from stress, and ensures more **reliable movement** in variable terrain conditions.

SOFTWARE

HARDWARE MANAGING (ARDUINO)



TARGET RECOGNITION (PYTHON)

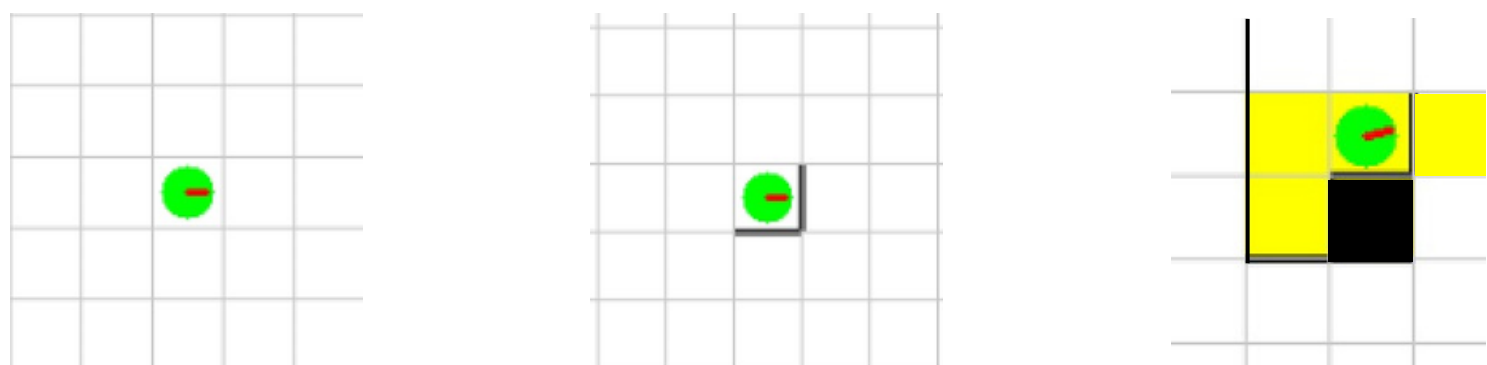


In our architecture, the **Arduino Mega** manages **low-level hardware** operations in real time, controlling the **motors** and polling **sensors** to guarantee **immediate responsiveness**. The only **exceptions** are the **color** and **infrared** reflection sensors, which are routed directly to the **Raspberry Pi 5** to feed its **high-level processing** and strategic **logic**.

To ensure **precise maneuverability**, a full **PID controller** executes stable **rotations** by **comparing target angles** with **real-time feedback**, while a **wall-alignment** routine dynamically calculates the robot's **heading error** by applying the **atan2** function to the **differential readings** of **two parallel side-facing ToF sensors**. Finally a dedicated **speed-ramping module** smoothly governs **acceleration** and **braking** to prevent wheel slippage and **minimize mechanical stress**.

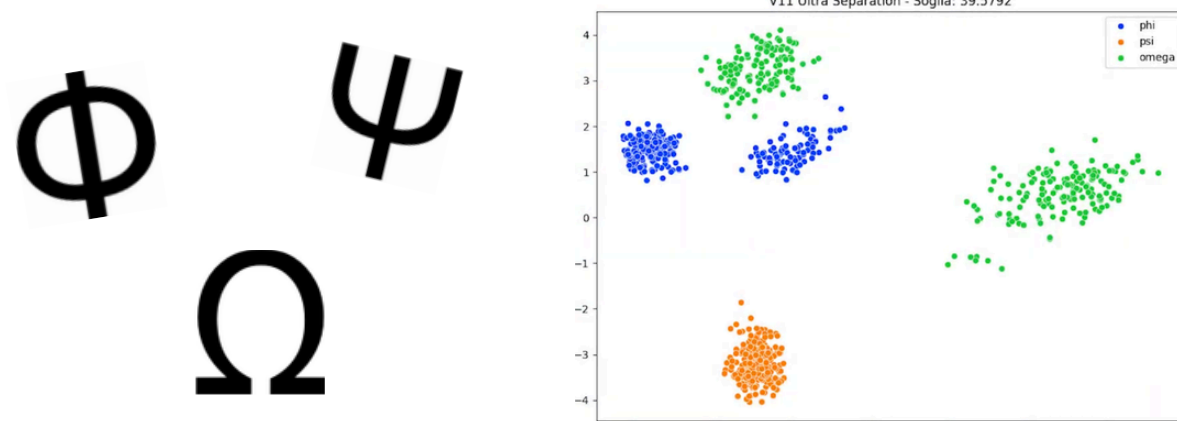
This integrated **Master-Slave paradigm** effectively balances deterministic hardware control with high computational power, ensuring highly **reliable autonomous navigation** even under complex arena conditions.

MAPPING (PYTHON)



The robot builds a **map of the environment** as it moves, updating it in **real time** using **sensor** and **camera** data. The area is divided into a **grid of cells**, where each cell stores information such as **explored zones**, **tile types**, and **walls**. Python handles **logging this data** and **updating the map** with every forward step, marking features encountered along the way. This enables the robot to **avoid previously visited areas**, choose more **efficient routes** using **Dijkstra** algorithm, and plan the optimal path to the goal. Ultimately, this **mapping system** is an essential tool for **autonomous navigation**, allowing the robot to **orient itself** and make **real-time decisions** based on exploration data.

KNN (PYTHON)



Once a visual region of interest (ROI) is isolated by the OpenMV camera, the **cropped frame** is instantly **streamed** to the **Raspberry Pi 5** via a dedicated communication interface. A specialized **Python-based K-Nearest Neighbors (KNN)** pipeline then executes **high-speed, rotation-invariant classification** on the detected blob. By **offloading** the initial **shape detection** to the edge co-processor, the Raspberry Pi's **CPU overhead is minimized**, dedicating its multi-threaded architecture strictly to classification and pathfinding. This **distributed approach** ensures deterministic **real-time processing**, allowing the master system to **instantly update** the maze navigation matrix or **execute** corresponding **victim-handling routines** without continuously halting the robot's exploration.

Modello 3D



PCB



FUTURA ELETTRONICA



Github

